Análise das interfaces atualizadas e do impacto na Hash Unit

Projeto: CI Digital Inatel, T25S2, Grupo 2. Responsável pelo Hash: Samuel. Data da análise: 13 de outubro de 2026.

O grupo está parcialmente alinhado: as larguras dos dados do Hash estão compatíveis com o novo diagrama, mas o RTL e os testes publicados de Hash e Flow Identification ainda utilizam `ready`. A decisão mais recente exige revisar o controle de transferência, os contratos e os testes. O cálculo do CRC pode ser preservado.

A comparação utilizou as decisões da reunião informadas por Samuel, o PDF anexado e o estado de `main` no commit `68f813c16da04374cb4239ac24203091c1d24620`. Foram lidos 16 arquivos de texto, o diagrama publicado e os dois pacotes ZIP de implementação. Os arquivos consultados foram conferidos com seus identificadores de blob. Esta é uma análise de compatibilidade e de ajustes necessários; não é uma implementação nova da Hash Unit.

1. Estabeleça qual diagrama é a referência.

O arquivo anexado `Interfaces_atualizado.pdf` é idêntico, byte a byte, ao arquivo `docs/images/Interfaces.pdf` publicado nesse commit. Portanto, o diagrama atualizado já está no GitHub. Seu blob é `d3de788bb7d5875b6da34fcc58ccc4c2a58306fb`.

Na página 1, a conexão FI -> Hash possui chave de 104 bits, comprimento de 16 bits, timestamp de 80 bits e `valid`. A saída Hash -> Controller acrescenta o índice de 16 bits. Na página 2, o sinal de validade dessa saída aparece com o nome `hash_valid`.

As indicações `[32]`, `[16]` e `[104]` do desenho representam larguras, não limites superiores de vetores SystemVerilog. No RTL, 32 bits são `[31:0]`, 16 bits são `[15:0]` e 104 bits são `[103:0]`.

Decisão da reunião	Situação do Hash preparado	Impacto
IP com 32 bits e porta com 16 bits	A chave já usa `32 + 32 + 16 + 16 + 8 = 104` bits.	Preservar a chave. Corrigir eventuais documentos antigos com outras larguras.
Índice hash com 16 bits	O RTL já entrega todos os 16 bits do CRC.	Preservar `hash_index_out[15:0]`.
Palavra da Flow Table com 32 bits	O Hash entrega um índice e metadados, não palavras de memória.	Alteração de memória não muda o CRC nem as portas de dados do Hash.
Transferência somente por `valid`	O Hash e a FI publicados ainda usam `ready`.	Revisar os dois módulos, seus contratos e testbenches.
Captura dos dados seguida de `start`	O Hash atual captura diretamente dentro de `IDLE`, sem `start`.	Separar recepção e acionamento do processamento.
Operação sem FIFO	O Hash atual já armazena um registro por vez, sem FIFO.	Preservar a orientação e documentar a folga exigida entre publicações.

2. Preserve a função do Hash e suas larguras.

A chave continua sendo `{src_ip, dst_ip, src_port, dst_port, protocol}`, sem troca de origem e destino. O CRC continua processando 13 bytes, começando em `flow_key[103:96]` e terminando em `[7:0]`.

O perfil usado no pacote publicado é CRC-16/CCITT-FALSE: polinômio `0x1021`, inicialização `0xFFFF`, sem reflexão e sem XOR final. As decisões recebidas não redefinem esse perfil. Manter esse cálculo conserva os valores de referência e a repetibilidade para a mesma chave.

O exemplo da FI permanece:

```
flow_key   = 104'hC0A8010AC0A80114C35001BB06;
hash_index = 16'hB511;
```

O índice foi recalculado com `binascii.crc_hqx` durante esta análise e continua `B511`. Comprimento e timestamp não participam do CRC, mas precisam continuar associados ao mesmo pacote.

Dado	Entrada no Hash	Saída do Hash
`flow_key`	104 bits	104 bits
`packet_length`	16 bits	16 bits
`timestamp`	80 bits	80 bits
`hash_index`	-	16 bits
Total de dados	200 bits	216 bits

A interface revisada pode preservar os nomes de dados com sufixos `_in` e `_out`. As duas portas a retirar são `in_ready` e `out_ready`. Sem acrescentar portas externas, o módulo passa de 13 para 11 portas. Clock e reset continuam necessários, mesmo que não estejam desenhados individualmente em cada bloco.

3. Substitua o handshake por uma publicação de um ciclo.

Na interface anterior, a aceitação dependia de `in_valid && in_ready`, e a entrega de `out_valid && out_ready`. Enquanto o consumidor estava indisponível, o Hash mantinha o resultado apresentado. Essas regras estão nos contratos e nos pacotes atuais, mas pertencem ao protocolo anterior.

Conforme a reunião, a versão revisada deverá capturar chave, comprimento e timestamp na borda em que `in_valid = 1`, com reset inativo. Quando o resultado completo estiver pronto, deverá apresentá-lo com `out_valid = 1` por um ciclo. Na próxima borda, o Controller captura esse registro por `hash_valid`.

Todos os campos devem estar estáveis durante a janela de validade e atender aos requisitos temporais da borda de captura. Um pulso de um ciclo inteiro permite uma captura síncrona; ele não significa que a função de hash ou o LOOKUP terminem em um ciclo.

Como o receptor verifica o nível de `valid` em cada borda, manter `valid = 1` por vários ciclos com o mesmo pacote pode gerar capturas repetidas. A nova versão não deve manter a validade aguardando um `ready` que deixou de existir.

Conexão	Ligação de dados e validade	Mudança de controle
Parser -> FI	`metadata_valid -> in_valid`, junto dos campos extraídos.	Preservar a publicação por validade.
FI -> Hash	`FI.out_valid -> Hash.in_valid`, junto de chave, comprimento e timestamp.	Retirar `Hash.in_ready -> FI.out_ready`.
Hash -> Controller	`Hash.out_valid -> Controller.hash_valid`, junto dos quatro campos de saída.	Retirar a dependência de `Hash.out_ready`; o Controller publicado não possui porta `in_ready`.

Os nomes `valid`, `metadata_valid`, `out_valid` e `hash_valid` podem ser aliases de conexão. O módulo de integração deve ligar as portas reais; nomes diferentes, com o mesmo significado e largura, não constituem por si só um erro.

A retirada de `ready` no caminho de metadados não elimina os sinais de conclusão que continuam no diagrama: `rd_valid`, `wr_done` e `fin_done` ainda têm funções próprias na consulta à memória e na finalização.

4. Implemente `start` como controle interno.

O padrão informado pela reunião é: receber `valid`, capturar os dados e gerar `start`; depois, `start` inicia a FSM. O Controller atual já utiliza esse padrão em dois blocos sequenciais. Seu `start` é interno, não uma porta entre módulos.

O Hash deve manter essa mesma distinção. `in_valid` informa a chegada de um registro; `start` informa à lógica interna que o registro capturado deve ser processado; `out_valid` informa ao Controller que o resultado está pronto. São três momentos diferentes.

O RTL anterior guarda a chave em `key_reg` e começa o processamento diretamente no estado `IDLE`. Ao separar recepção e FSM, é preciso garantir que a chave usada no cálculo e os metadados associados não sejam sobrescritos por uma nova publicação durante uma operação.

Momento, em uma organização possível	Comportamento
Borda E0	`in_valid = 1`: captura os 200 bits e registra `start = 1`.
Borda E1	A FSM observa o `start` registrado e inicia a operação; a lógica de recepção limpa o pulso.
Bordas de cálculo	Atualiza o CRC a partir da chave armazenada, um byte por atualização.
Resultado disponível	Publica índice, chave e metadados com validade de um ciclo.

Com atribuições não bloqueantes, um bloco sequencial observa na borda o valor anterior de `start`. Portanto, gerar `start <= 1` em um bloco e usá-lo em outro não faz a FSM observar o novo valor naquela mesma borda.

Os 13 passos de cálculo de bytes continuam aplicáveis, mas a latência total precisa ser medida na versão revisada. Se a inicialização que antes ocorria em E0 for deslocada para E1 e o primeiro byte continuar no ciclo seguinte, a disponibilidade passa de E13 para E14. Esse é um exemplo de organização, não um resultado medido de um novo RTL. O número antigo não deve ser copiado automaticamente para o novo testbench.

5. Registre a condição temporal que permite operar sem `ready`.

Há uma correção técnica na justificativa da reunião: usar o mesmo clock define as bordas de amostragem, mas não garante que uma FSM esteja livre quando um novo pacote chegar. `ready` também é utilizado para controlar capacidade em circuitos com um único clock. A documentação AMD PG038 apresenta um exemplo de interface com clock único e `valid/ready`; PG104 também documenta um modo sem `ready`, que opera sem capacidade de bloquear o produtor.

A decisão de usar somente `valid` pode ser mantida na PoC. A justificativa adequada é a folga entre publicações, relatada pelo professor, em relação ao tempo necessário para atender cada registro. Essa folga precisa cobrir tanto o Hash quanto o Controller. Isso não exige adicionar FIFO nesta análise.

O contrato deverá fixar um intervalo admissível entre registros e demonstrar que cada destinatário estará livre nas bordas de publicação. O intervalo deve considerar o pior caso admitido para a etapa correspondente; latência do resultado e intervalo entre aceitações são grandezas diferentes.

Foi realizado um teste dirigido com o `controller_base.sv` publicado, sem alterar seu código. O teste aplicou o índice 5, esperou a FSM sair de `IDLE` e publicou o índice 100 durante o LOOKUP. O modelo de resposta de memória usou a mesma latência do testbench publicado do Controller.

```
OBSERVED captures=2 lookup_starts=1 read_requests=5
final_hash_index_reg=100
CONFIRMED: segundo pulso foi capturado,
mas nao iniciou outro LOOKUP.
```

O primeiro índice iniciou cinco leituras nos endereços 80 a 84. O segundo índice entrou no registrador de recepção, mas seu pulso de `start` desapareceu enquanto a FSM estava ocupada. A execução, em Verilator, terminou em 466 ns e confirmou esse comportamento.

Esse cenário foi aplicado deliberadamente fora da condição de disponibilidade da PoC. Ele não demonstra perda com o tráfego real espaçado; demonstra que a garantia de folga não pode ser substituída apenas pela existência de clock comum ou por um pulso de validade.

Uma checagem de simulação que denuncie publicações fora da capacidade prevista ajuda a verificar o contrato sem adicionar portas de controle. O próximo teste do Hash deve exercitar o intervalo mínimo acordado e detectar sua violação, preservando o registro em cálculo.

6. Separe o índice lógico do endereço de memória.

A Flow Table atual armazena 16 palavras de 32 bits por registro: `16 × 32 = 512` bits. Com 65.536 registros, há `2^20` palavras endereçáveis, exigindo endereço físico de 20 bits. Essa organização aparece em `Flow_Table.md`, `Flow_Table_interface.md` e no RTL `flow_table.sv`.

O Hash produz o índice lógico de 16 bits. O Controller calcula:

```
base_addr = hash_index × 16
word_addr = base_addr + word_offset
```

Para o exemplo `hash_index = 0xB511`, a base é `0xB5110`. O Hash continua entregando `B511`; ele não precisa passar a entregar um CRC de 32 bits ou calcular os endereços das palavras.

O Controller atual declara `rd_addr[20:0]`, com 21 bits, e calcula `{1'b0, hash_index_reg, 4'b0000}`. O bit extra permanece zero no cálculo atual. Assim, há um desalinhamento de largura com a memória de 20 bits, mas não um erro matemático nesse endereço-base. As equipes

devem unificar a porta em 20 bits ou explicitar a adaptação dos 20 bits inferiores e a condição de bit superior zero.

7. Confira a sintonia entre os arquivos efetivamente publicados.

Tema	Evidência encontrada	Ação e responsável
Diagrama	Anexo idêntico ao PDF do GitHub; índice 16, palavra 32, timestamp 80, sem `ready`.	Usar essa referência comum. Equipes.
Hash	ZIP contém RTL com `in_ready`, `out_ready` e estado `RESULT` aguardando aceitação. Guia e contrato mantêm o handshake.	Revisar controle, contrato, guia e testes. Samuel.
FI	ZIP contém `out_ready` e retenção de `out_valid` enquanto bloqueada. A implementação é uma etapa registrada sem `start` ou FSM.	Retirar dependência de `ready` e acompanhar o padrão de recepção acordado. Bruno.
Parser, larguras da chave	RTL usa IP 32, porta 16, protocolo 8 e pulso de `metadata_valid`.	Compatível nesses itens. Herman.
Parser, timestamp	Contrato indica 80 bits; o parâmetro padrão do RTL é `TS_WIDTH = 64`.	Instanciar/configurar 80 bits e ajustar o testbench; ou documentar uma adaptação de formato aprovada. Herman e verificação.
Controller, recepção	RTL usa `hash_valid`, captura os quatro campos e gera `start`; não tem `ready`.	Compatível com o padrão novo, sob a condição temporal documentada. Equipe do Controller.
Flow Table	Contrato e RTL usam dados de 32 bits, endereço de 20 bits e 16 palavras por registro.	Compatível com a decisão de memória. Equipe da Flow Table.
README e contrato do Controller	README ainda descreve memória de 16 bits e 32 palavras por registro; contrato do Controller ainda declara `rd_data` e `wr_data` de 16 bits.	Atualizar para a organização adotada. Equipes de features/documentação.
Endereço do Controller	RTL e sua documentação usam 21 bits; diagrama e Flow Table usam 20.	Uniformizar ou explicitar a adaptação segura. Equipes do Controller e Flow Table.
Reset	Parser: ativo baixo assíncrono. FI/Hash: ativo baixo síncrono. Controller/Flow Table: ativo alto síncrono.	Fixar polaridade, modo e conexões no módulo de integração. Equipes.
Finalizer	Documento cita timestamps de 48 bits e ainda discute `ready`; diagrama e caminho de metadados indicam 80 bits.	Atualizar a especificação de timestamp e protocolo. Equipe do Finalizer.

O RTL do Hash está no arquivo `semana4_hash_unit_SAMUEL.zip`, e o RTL da FI está em `semana4_fiu_BRUNO.zip`, ambos em `Especificação/Extracao de Metadados/`. As pastas `rtl/hash_unit/` e `rtl/flow_identification/` ainda têm somente `.gitkeep`, mas isso não significa ausência das implementações: elas foram encontradas e inspecionadas nos ZIPs. Disponibilizar também os fontes separados facilitará revisão de diferenças e integração no GitHub.

A mistura de nomes e polaridades de reset não deve ser resolvida apenas renomeando `reset_n` para `rst`. Um reset ativo baixo exige valor zero para reinicializar; um reset ativo alto exige valor um. O diagrama não define essa polaridade. A integração precisa fornecer os níveis corretos a cada bloco e registrar o comportamento temporal do reset.

Há ainda um ponto indireto nas features: a memória é descrita com registros físicos de 512 bits, enquanto `flow_entry` no diagrama e no Controller possui 384 bits. Isso pode ser válido com extração e empacotamento de campos, mas esse mapeamento completo ainda precisa ser

explicitado. No LOOKUP atual, somente W0 a W4, totalizando 160 bits, são preenchidas em `flow_entry`; o restante fica zero. O Hash não deve alterar sua chave ou seu timestamp para acomodar essa diferença.

8. Revise os arquivos do Hash na ordem adequada.

Ordem	Arquivo ou conjunto	Ajuste necessário
1	Contrato revisado da Hash Unit	Retirar as duas portas `ready`; definir validade de um ciclo, recepção atômica e condição temporal. Registrar clock/reset e `start` interno.
2	`rtl/hash_unit.sv` no pacote	Preservar CRC e dados; separar recepção e FSM; publicar um resultado por pulso; retirar espera pelo Controller.
3	`tb/tb_hash_unit.sv`	Contar eventos por validade, verificar largura do pulso, associação, reset, latência nova e espaçamento admissível.
4	Referência Python, CSV e MEM	Reutilizar os valores do CRC: o perfil e os 216 bits do registro de conferência continuam iguais.
5	Guia da Semana 4 e registro de validação	Atualizar portas, sequência temporal e critérios; executar novamente os testes antes de declarar a versão nova validada.
6	Relatórios de contratos da Semana 3	Registrar a decisão mais recente como revisão, distinguindo versões históricas das que passam a orientar a integração.

Os testes antigos de espera por `ready` deixam de ser critérios do novo protocolo. Os casos de CRC, repetição de chave com metadados diferentes, preservação dos 80 bits do timestamp e cancelamento por reset continuam úteis. O resultado anterior de 181 vetores não comprova uma interface que ainda será modificada; ele comprova a versão anterior e fornece a referência para a nova execução.

Antes da conexão final, devem estar definidos o intervalo mínimo de publicações, a latência da implementação revisada, o reset compartilhado e a representação do timestamp. O Parser da PoC usa microssegundos inteiros; ter 80 bits de largura não define, sozinho, a unidade ou o formato temporal. O Hash deve preservar o valor recebido, sem converter esse formato.

A revisão necessária à sua parte é de controle e integração: a chave, o perfil CRC e o índice completo de 16 bits podem ser mantidos. Bruno precisa acompanhar a mudança do lado da FI, e as equipes de features precisam atualizar os contratos que ainda descrevem a memória anterior.

As fontes abaixo estão vinculadas ao commit analisado, para que o parecer possa ser conferido mesmo após novas alterações em `main`.

- Diagrama de interfaces publicado: duas páginas; arquivo idêntico ao anexo recebido.
- Contrato Hash revisado sem FIFO e guia Hash da Semana 4: protocolo anterior e parâmetros do CRC.
- Pacote Hash de Samuel, blob `c0f3cf34a8c17b3acc9b74f0c7469a6183b1ff9e`: `semana4_hash/rtl/hash_unit.sv` e seu testbench foram inspecionados.
- Pacote FI de Bruno, blob `034b27f485c3b3f22990d70b980fc6cf41ca7703`: `semana4_fiu_BRUNO/rtl/flow_identification_unit.sv` e seu testbench foram inspecionados.
- Guia FI da Semana 4: operação registrada e dependência de `out_ready`.
- Contrato do Parser e RTL da PoC: larguras, pulso de validade, parâmetro de timestamp e reset.
- Contrato do Controller, RTL do Controller e documentação do LOOKUP: recepção, `start`, endereçamento, dados e limites do teste publicado.
- Especificação da Flow Table, contrato da memória e RTL da memória: palavras de 32 bits, 16 palavras por entrada e endereço de 20 bits.
- Documento do Flow Finalizer e README do projeto: pontos de documentação que ainda precisam de atualização.
- AMD PG038 - AXI Virtual FIFO Controller, seções Clocking e I/O Signals: exemplo técnico de clock único com controle `valid/ready`.
- AMD PG104 - NonBlocking Mode: exemplo técnico de operação sem `ready` e sem bloqueio do produtor.